ASSIGNMENT OS_LAB_07

Course code: CC-311 LAB

Submitted by: Ramsha Arooj

Submitted to: Sir Kamran

Roll Number: 110763

Department: Information Technology

Semester: 5th

Date:12/12/2025

PTHREAD_CREATE FUNCTION

TASK#01:

Q. Program to create threads in Linux. Thread prints 0-4 while the main process prints 20-24.

Program:

```
#include <stdio.h>
#include <pthread.h>
void* mythreadfun(void* arg){
        printf("thread is running...\n");
        return NULL;
        }
int main(){
        pthread_t tid;
        if(pthread_create(&tid, NULL, mythreadfun, NULL)!=0){
                printf("error creating thread\n");
                return 1;
        }
        pthread_join(tid, NULL);
        printf("thread completed.\n");
        return 0;
}
```

Output:

```
ramsha@myVM:~$ gedit task1.c
ramsha@myVM:~$ gcc -pthread task1.c -o task1
ramsha@myVM:~$ ./task1
Thread is running.
thread completed.
ramsha@myVM:~$
```

Task#02:

Program to create a thread. The thread prints numbers from zero to n, where value of n is passed from the main process to the thread. The main process also waits for the thread to finish first and then prints from 20-24.

Program:

```c
1 #include <stdio.h>
2 #include <pthread.h>
3 void* printfnumbers(void* arg) {
4       int n = *(int*)arg;
5
6       for (int i=0; i<n; i++) {
7           printf("thread: %d\n",i);
8
9       }
10
11      return NULL;
12 }
13
14 int main() {
15     pthread_t thread;
16     int n;
17
18     printf("Enter value of n: ");
19     scanf("%d",&n);
20
21     pthread_create(&thread,NULL,printfnumbers,&n);
22
23     pthread_join(thread,NULL);
```

```
    for (int i=20; i<24; i++) {
        printf("main: %d\n", i);
    }
     return 0;
}
```

Output:

```
Help
ramsha@myVM:~$ gcc task2.c -o task2 -pthread
ramsha@myVM:~$ ./task2
Enter value of n:5
Thread: 0
Thread: 1
Thread: 2
Thread: 3
Thread: 4
main:20
main:21
main:22
main:23
ramsha@myVM:~$
```

Task#03:

Program to create a thread. The thread is passed more than one input from the main process. For passing multiple inputs we need to create structure and include all the variables that are to be passed in this structure.

```
#include <stdio.h>
#include <pthread.h>

struct data {
    int start;
    int end;
    char message[50];

};

```

```c
void* threadfunction(void* agr) {
        struct data* d = (struct data*)agr;

        printf("Message from thread: %s\n",d->message);

        for (int i=d->start; i<= d->end; i++) {

             printf("thread: %d\n",i);

        }

        return NULL;
}

int main() {
       pthread_t thread;
       struct data d;

       printf("Enter start number: ");
       scanf("%d", &d.start);

       printf("Enter end number: ");
       scanf("%d", &d.end);

       printf("Enter a message for thread: ");
       scanf(" %[^\n]s",d.message);

       pthread_create(&thread,NULL,threadfunction, &d);

       pthread_join(thread,NULL);

       printf("main thread work finished.\n");

        return 0;
}
```

Output:

```
ramsha@myVM:~$ gcc task2.c -o task2 -pthread
ramsha@myVM:~$ ./task2
Enter value of n:5
Thread: 0
Thread: 1
Thread: 2
Thread: 3
Thread: 4
main:20
m Ubuntu Software
main:22
main:23
ramsha@myVM:~$ gedit task3.c
ramsha@myVM:~$ gcc task3.c -o task3 -pthread
ramsha@myVM:~$ ./task3
Enter start number:3
Enter end number:8
Enter a message for the thread:Message from thread:
Thread: 3
Thread: 4
Thread: 5
Thread: 6
Thread: 7
Thread: 8
Main thread work finished.
ramsha@myVM:~$
```

VIVA QUESTIONS:

Q1. Why is the second parameter in pthread_create() function generally taken as NULL?

The second parameter specifies thread attributes (like stack size, scheduling policy).

If we don't need special attributes, we pass NULL to use default attributes.

Q2. What is the significance of using pthread_join() function?

It waits for a thread to finish execution.

Ensures that the main thread can retrieve the thread's return value.

Prevents zombie threads.

Q3. Which function is used to return some value from the child process to the parent process?

In threads, the function is pthread_exit(void *retval).

The parent can retrieve it using pthread_join().

Q4. How can we pass multiple values to a thread?

Combine values in a structure and pass the structure's pointer as the thread argument.

Q5. How is the value returned by a thread saved/stored by the main process?

The main thread passes a void pointer to pthread_join(), which stores the thread's return value.